CINECA AGENTIC PLATFORM

An Enterprise-Grade Multi-Tenant AI Orchestrator with MCP Tools and Secure Graph Querying

University:
Sapienza University of Rome

Department:
Information Engineering, Electronics and Telecommunications

Academic Year:
2025/2026

University Advisor:
Prof. Maria De Marsico

Company:
CINECA

Author:
Arman Feili

University Co-Advisors:
Prof. Marco Raoul Marini
Dr. Valerio Venanzi

Cineca Advisors:
Dr. Giuseppe Melfi
Dr. Marco Puccini

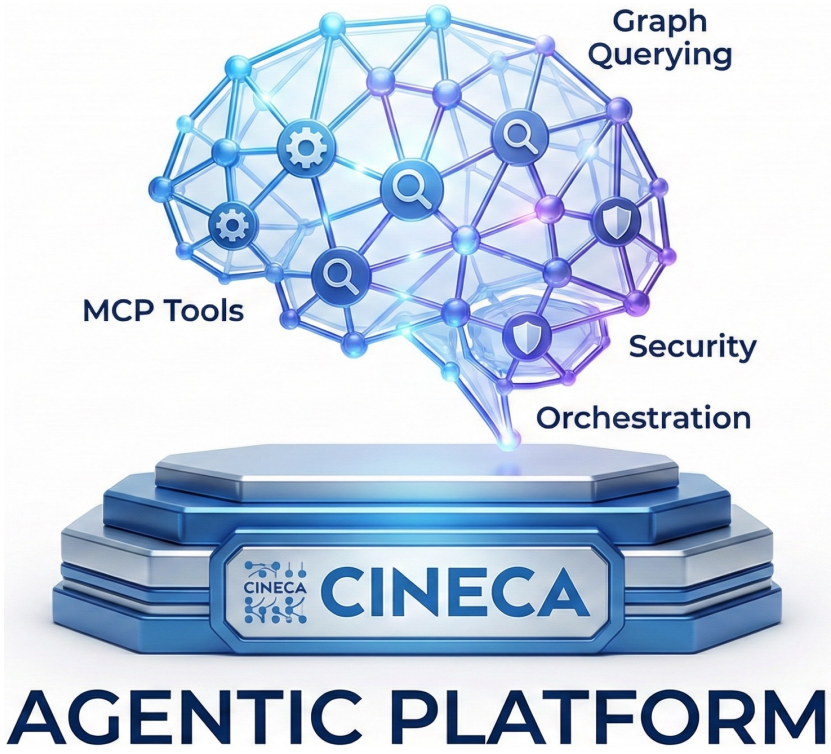
SAPIENZA
UNIVERSITÀ DI ROMA

CINECA

INDEX

Overview of the presentation structure

- Production Gap
- Final Architecture (Simplified + Full)
- What Was Delivered
- Authentication Layer & UI
- Security Middleware Stack
- API Layer - Routers
- Job Creation
- Workers + Agent.run Handler → Job Processing Engine
- Service Layer → Orchestrator Service
- LLM Providers → Model-Agnostic Architecture
- MCP Runtime & Tools - 34 Tools, 12 Categories
- Data Layer → Redis, PostgreSQL, Memgraph
- Adapters + Resilience
- Background Tasks + Observability
- Comparison with the State of the Art
- Conclusion + Future Work



PRODUCTION GAP

The gap between what users need and what current tools provide.

The Problem

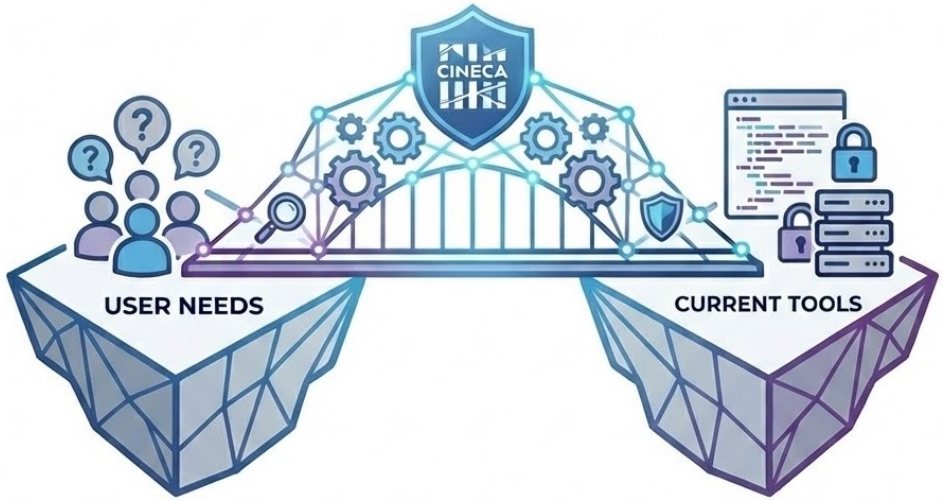
- Most users cannot write Cypher, so they depend on developers to access graph data. This creates delays and keeps data locked behind technical complexity.

Why Simple Chatbots Are Not Enough

- Basic chatbots can answer questions, but they cannot
  - handle multi-step tasks
  - query databases or run jobs
  - enforce security and access control
  - provide audit trails and traceability

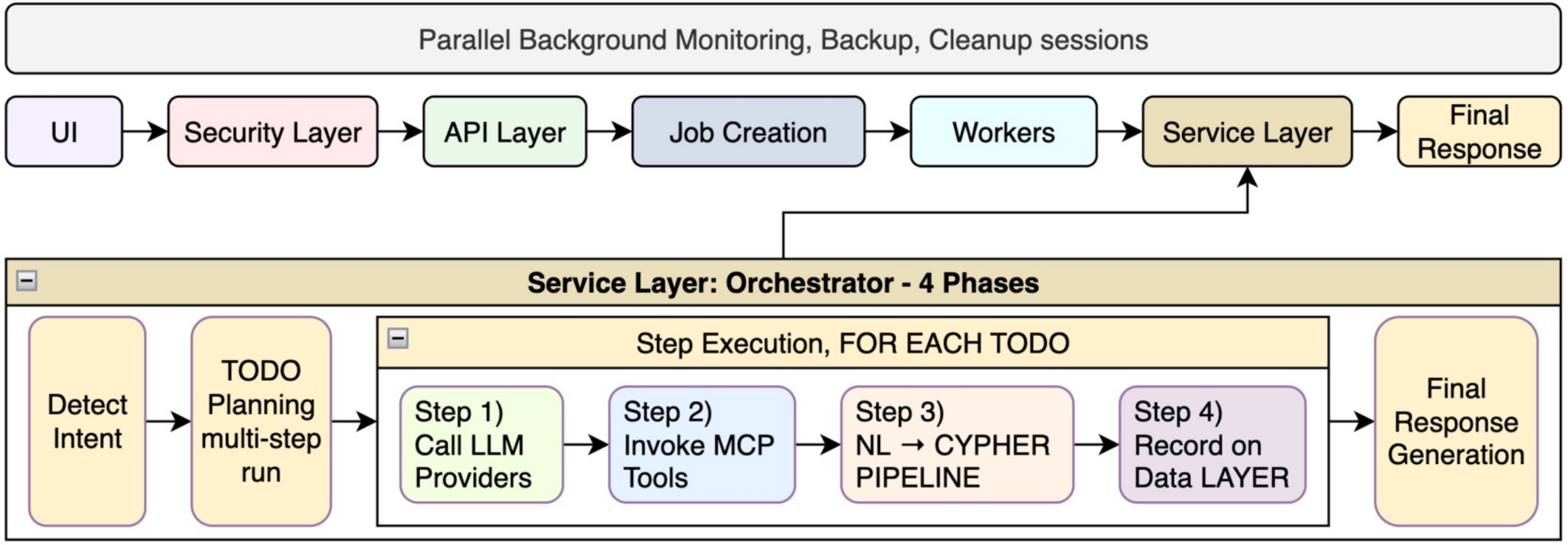
Why This Matters for CINECA

- CINECA needs a platform that lets users query graph data directly in natural language, while remaining:
  - Secure
  - Scalable
  - Reproducible
  - cost-efficient
  - flexible across LLM providers



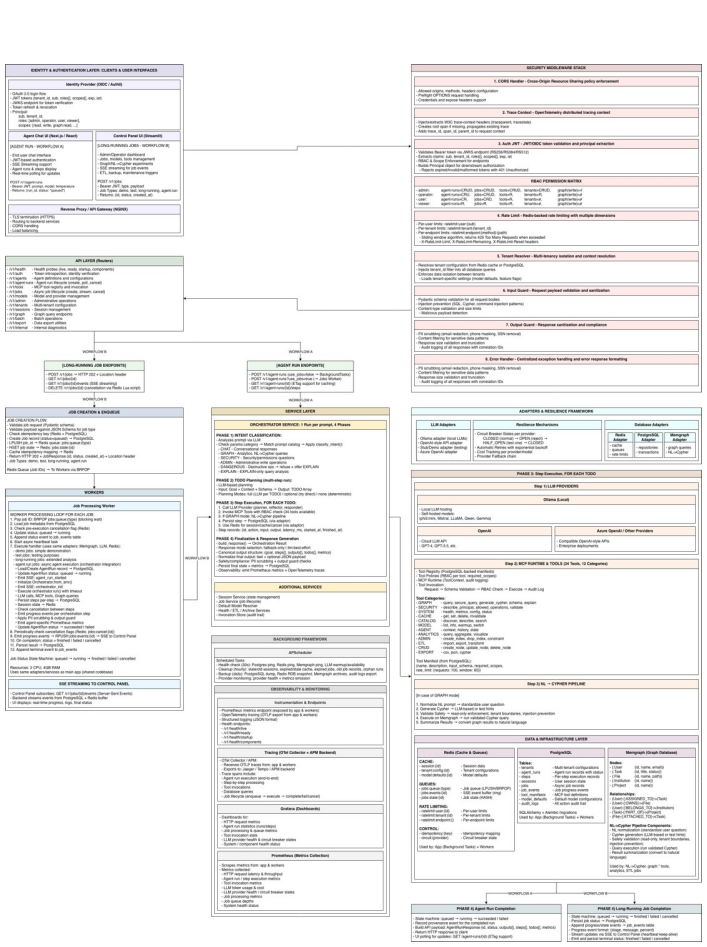
FINAL ARCHITECTURE

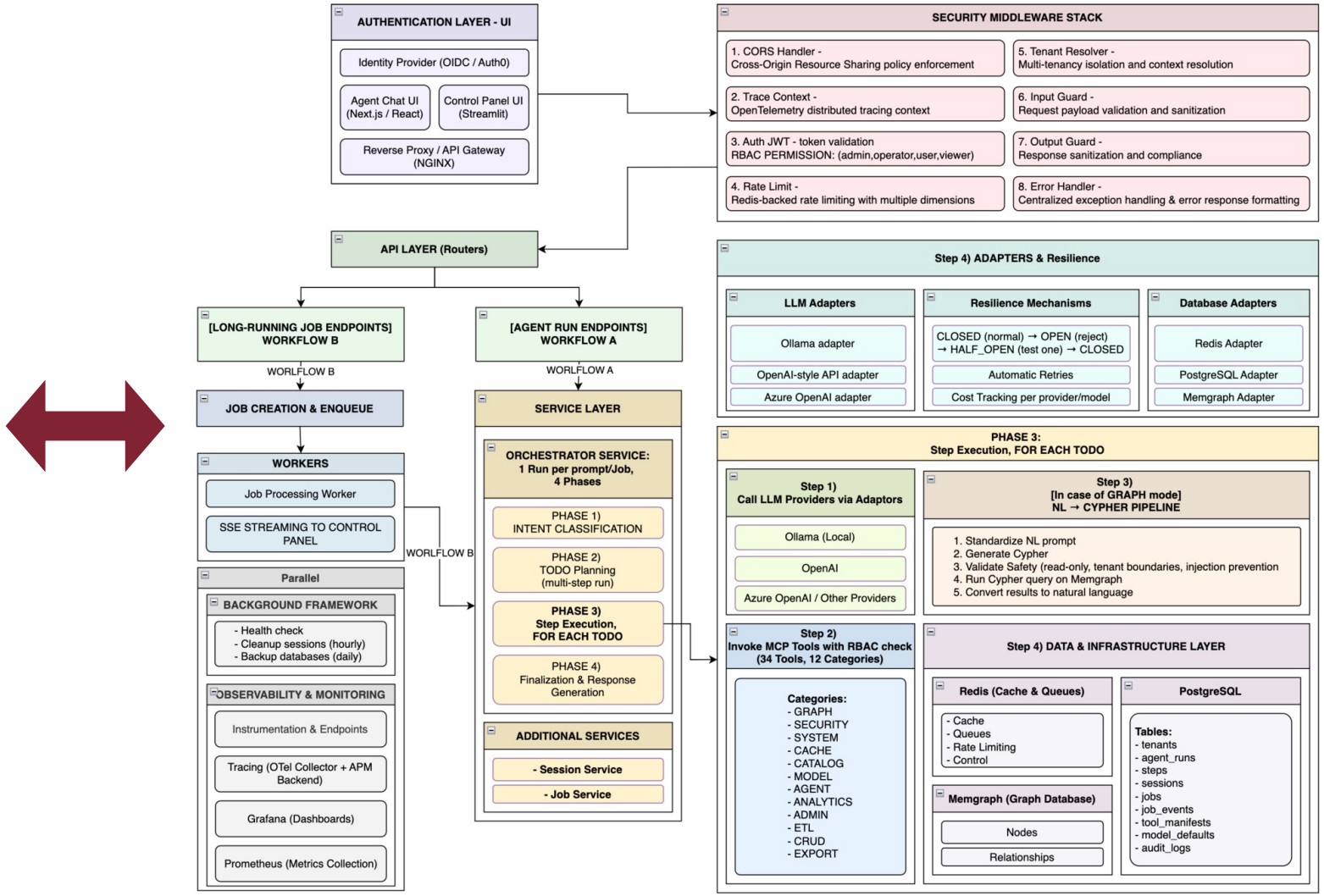
Simplified



FINAL ARCHITECTURE

Full





WHAT WAS DELIVERED

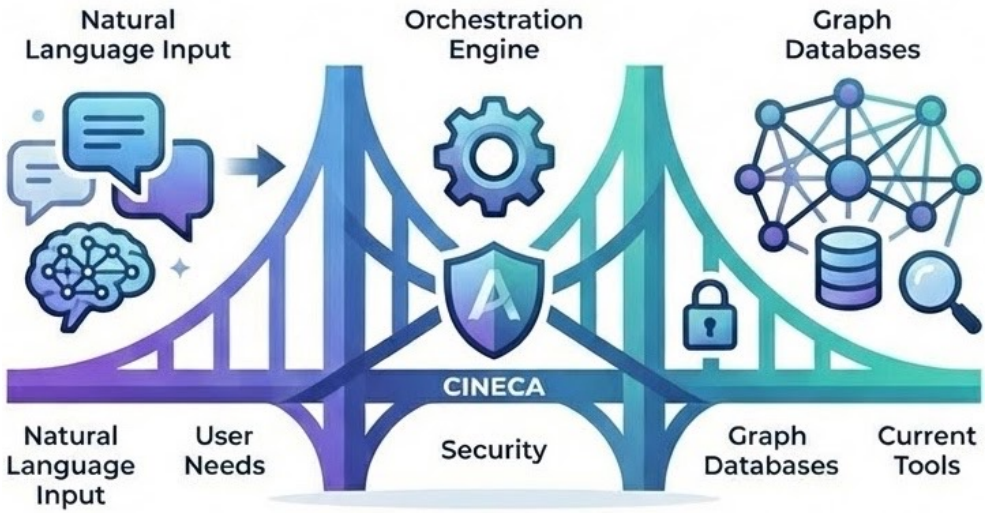
A production-ready agentic platform for secure natural-language access to graph data.

Core Delivery

- End-to-end platform: UI, APIs, orchestration, workers, and a three-database backend
- Safe 4-phase agent workflow with NL→Cypher capabilities
- Multi-tenant security and support for multiple LLM providers
- Reliability, background processing, and observability built in

Production Evidence

- 16+ platform components (databases, queues, gateway, tracing)
- 76 API endpoints across 16 router groups (e.g. auth, tools, jobs, graph)
- 34 tools across 17 categories with governed execution
- 3,000+ automated tests (unit, integration, security, end-to-end)
- ~411,700 total lines of code



AUTHENTICATION LAYER & UI

Users sign in through an identity provider, receive a JWT, and access the platform through a secure gateway.

Authentication & Access

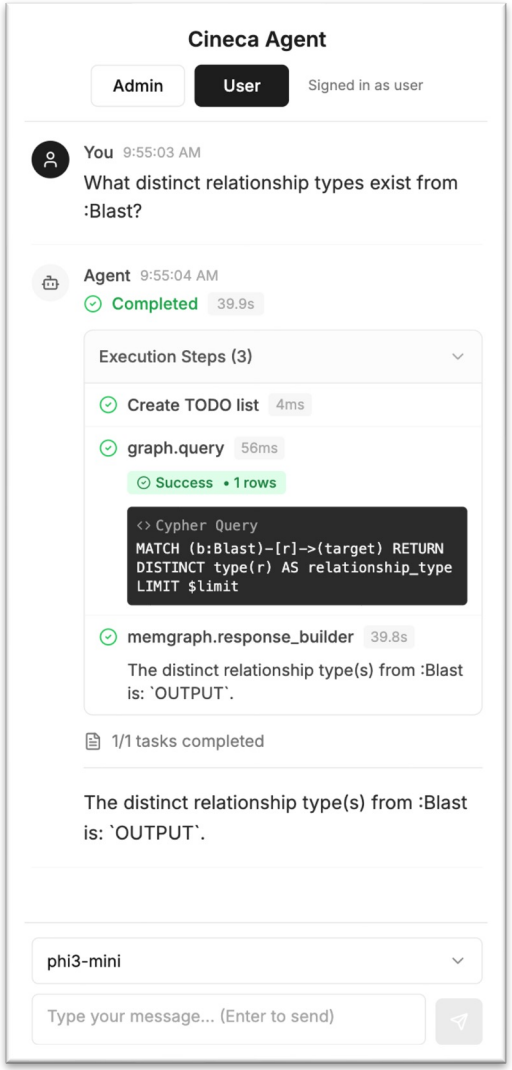
- Users log in with OAuth 2.0 and receive a JWT with their identity, tenant, role, and permissions
- Every request verifies the token and applies role-based access control
- The UI adapts to different access levels

Agent Chat UI

- Main interface for asking questions in natural language and interacting with agents
- Sends authenticated requests to the backend
- Shows live progress, execution steps, final results, and the selected model

Gateway

- NGINX handles secure access, routing, and CORS



SECURITY MIDDLEWARE STACK (Part 1/2)

Every request passes through 8 Security middleware layers before reaching the business logic.

1. CORS Handler
Controls which websites can call the API.
- Example: requests from the official chat UI are allowed; unknown domains are blocked.

2. Trace Context
Assigns a trace ID so one request can be followed across the whole system.
- Example: one user request can be tracked through the API, orchestrator, and database.

3. JWT Auth + RBAC
Checks the user token and enforces role-based permissions.
- Example: a viewer tries to delete a run and gets 403 Forbidden.

4. Rate Limiter
Prevents abuse by limiting how many requests can be sent.
- Example: after too many requests in one minute, the user gets 429 Too Many Requests.

Role	agent-runs	jobs	tools	tenants	Graph (write)
admin	CRUD	CRUD	CRUD	CRUD	✓
operator	CRU	CRUD	R	R	×
user	CR	CRD	R	×	×
viewer	R	R	R	×	×

SECURITY MIDDLEWARE STACK (Part 2/2)

Every request passes through 8 Security middleware layers before reaching the business logic.

5. Tenant Resolver
Ensures each user only accesses data from their own organization.
- Example: a user from one tenant cannot see data from another tenant.

6. Input Guard
Validates requests and blocks unsafe or malformed input.
- Example: an injection-like prompt is rejected before execution.

7. Output Guard
Cleans responses before they are returned to the user.
- Example: an email address in the output is redacted.

8. Error Handler
Returns safe, standardized errors without exposing internal details.
- Example: a database failure returns a generic error plus a correlation ID.

API LAYER - Routers

The platform exposes 76 endpoints grouped by domain under versioned APIs.

Core API Groups

- /v1/health → service and dependency health checks
- /v1/auth → authentication and token validation
- /v1/agents → agent definitions and configuration
- /v1/tenants → tenant settings, limits, and feature control
- /v1/models → LLM provider and model management
- /v1/tools → MCP tool discovery and invocation

Execution APIs

- /v1/jobs → long-running jobs, live progress, and cancellation
- /v1/agent-runs → start runs, track status, get results, cancel execution
- /v1/graph → direct Cypher queries and NL→Cypher execution
- /v1/sessions → session and conversation state management

Operational APIs

- /v1/admin → maintenance tasks, cache control, and system stats
- /v1/export → export runs, job results, and audit logs

JOB CREATION

Client submits a long-running task → the system validates it, avoids duplicates, and queues it safely.

1. Request Validation
- Checks required fields and payload structure
- Example: agent.run must include prompt, user_id, and tenant_id

2. Idempotency Check
- Looks for an existing job with the same idempotency key
- Example: if the client retries after a timeout, the same job is returned

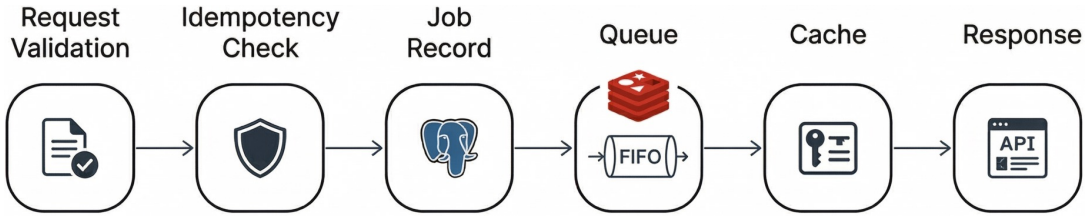
3. Job Record Creation
- Creates a new PostgreSQL job record with status = queued
- Example: the job is stored before execution starts

Supported Job Types
- demo, test, long-running, agent.run

4. Queue Enqueue
- Pushes the job ID into the Redis queue for workers
- Example: agent.run jobs go to jobs:queue:agent.run

5. Idempotency Cache
- Stores key → job ID mapping in Redis for fast duplicate detection
- Example: future retries can return the same job immediately

6. HTTP Response
- Returns 202 Accepted with the job ID and status
- Example: client receives a Location header to poll /v1/jobs/{id}



WORKERS + AGENT.RUN HANDLER → JOB PROCESSING ENGINE (Part 1/2)

Background workers pick up queued jobs and execute full agent workflows reliably outside the API process.

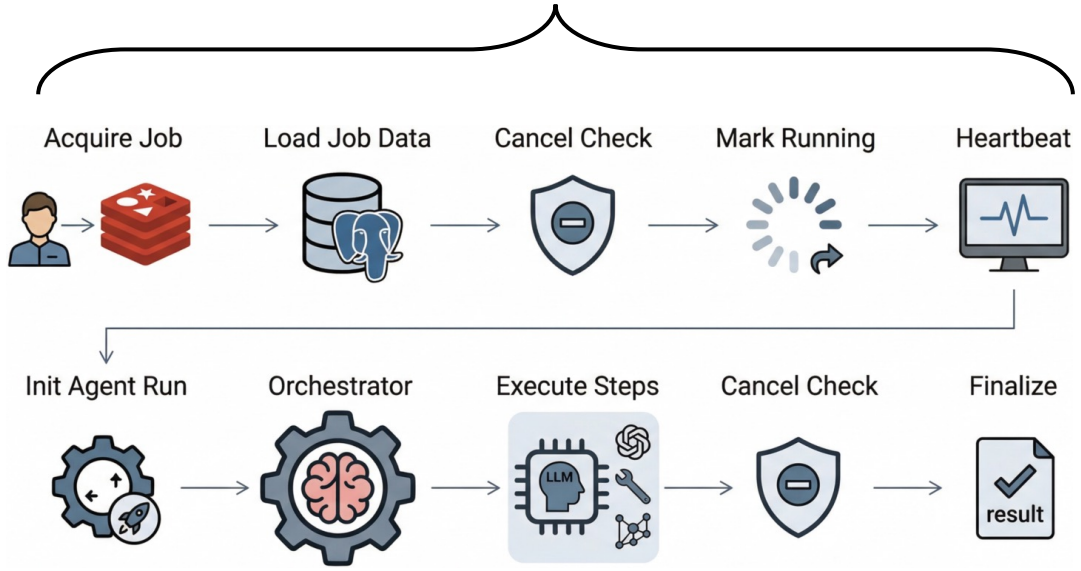
1. Acquire Job
- The worker waits for a job in Redis and takes exclusive ownership when one appears
- This prevents the same job from being processed twice

2. Load Job Data
- Using the job ID, the worker loads the full job record from PostgreSQL
- PostgreSQL is the source of truth; Redis only provides the queue entry

3. Check Early Cancellation
- Before starting, the worker checks whether the job was cancelled
- If already cancelled, execution is skipped safely

4. Mark as Running
- The worker updates the job state from queued to running
- This state change is stored for tracking, audit, and live updates

5. Start Heartbeat
- While the job is active, the worker sends periodic heartbeat updates
- If the heartbeat stops, the system can detect a stuck or failed worker



WORKERS + AGENT.RUN HANDLER → JOB PROCESSING ENGINE (Part 2/2)

Background workers pick up queued jobs and execute full agent workflows reliably outside the API process.

6. Initialize Agent Run

- For agent.run jobs, the worker creates an AgentRun record linked to the job
- Startup events are emitted so the UI can show immediate progress

7. Run the Orchestrator

- The worker calls the orchestrator as the core execution engine
- The orchestrator plans and executes the workflow step by step

8. Execute and Persist Steps

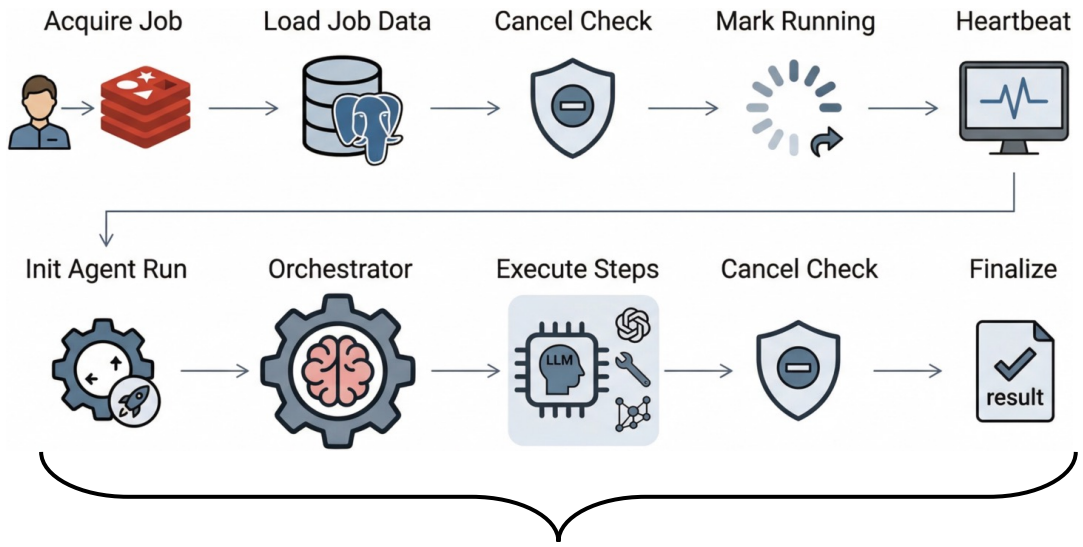
- Each step may call an LLM, invoke MCP tools, or query the graph
- Completed steps are saved immediately so progress is not lost

9. Check Cancellation During Execution

- Between steps, the worker checks again whether cancellation was requested
- If cancelled, it stops cleanly after the current completed step

10. Emit Progress and Finalize

- Progress events are streamed live, outputs are post-processed for safety, and metrics are recorded
- At the end, the final result is saved and the job is marked as succeeded or failed



SERVICE LAYER → ORCHESTRATOR SERVICE (Part 1/3)

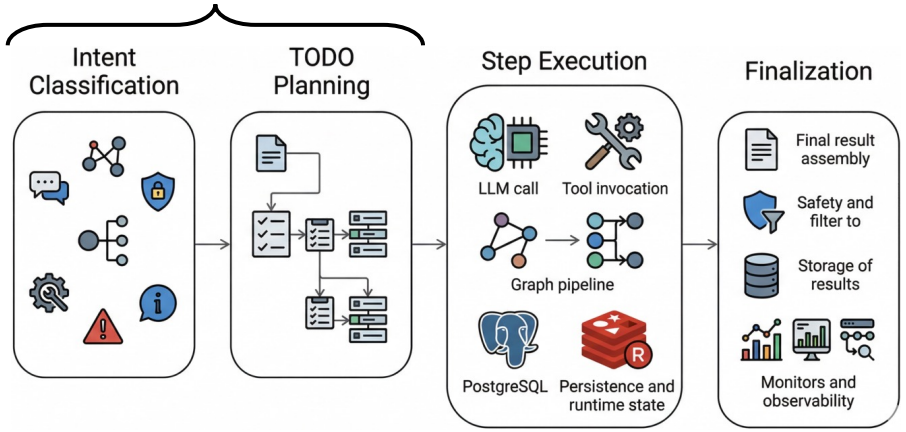
The orchestrator contains the core business logic and processes each prompt in 4 phases.

Phase 1 → Intent Classification

- The orchestrator first determines what kind of request the user is making
- It checks a prompt catalog first; if no match is found, it uses an LLM classifier

Main intent types include:

- CHAT → normal conversational response
- GRAPH → NL→Cypher query
- SECURITY → permission or access question
- ADMIN → administrative action
- DANGEROUS → destructive request, refused safely
- EXPLAIN → explain an action without executing it



Phase 2 → TODO Planning

- For multi-step tasks, the orchestrator turns the prompt into an ordered TODO plan
- Planning uses the user request, conversation context, and available schema/tool information
- The output is a structured sequence of actions, such as query → analyze → summarize

Planning Modes:

- full → creates a detailed step-by-step plan for complex tasks
- optional → tries direct execution first, plans only if needed
- none → uses a fixed path for predefined or catalog-matched requests

SERVICE LAYER → ORCHESTRATOR SERVICE (Part 2/3)

The orchestrator contains the core business logic and processes each prompt in 4 phases.

Phase 3 → Step Execution

The orchestrator executes the TODO plan step by step using LLMs, MCP tools, the graph pipeline, and runtime state.

1. Call LLM Providers

- Uses different LLM roles for planning, checking results, and generating responses
- Calls go through adapters with retries, fallbacks, and circuit breakers

2. Invoke MCP Tools

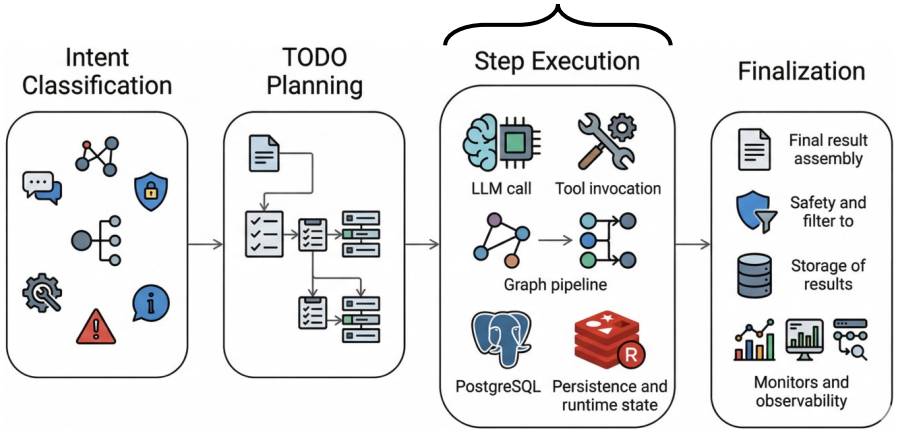
- Calls tools for graph queries, security checks, cache access, and export
- Each tool call is permission-checked and keeps session context

3. Run the Graph Pipeline

- For graph requests, the system runs a 6-stage NL→Cypher flow:
  - `normalize input → check catalog match → generate Cypher → validate safety → execute on Memgraph → summarize results.`
- Validation includes syntax, tenant boundaries, query depth, timeout, result limits, and read-only enforcement

4. Save Progress and Manage State

- Each step is saved to PostgreSQL for traceability and recovery
- Redis is used for caching and cancellation checks




```
"Show 5 Blast pairs outputting to same BlastedSeq"

1. Normalize → "blast pairs same blastedseq limit 5"
2. Catalog → No match → LLM
3. Generate →
MATCH (b1:Blast)-[:OUTPUT]->(s)<-[:OUTPUT]-(b2:Blast)
WHERE b1<>b2 LIMIT 5
4. Validate → ✓ Read-only ✓ Tenant ✓ Depth ✓ Timeout
5. Execute → Returns 5 pairs
6. Summarize →
"Found 5 Blast pairs sharing BlastedSeq targets..."
```

SERVICE LAYER → ORCHESTRATOR SERVICE (Part 3/3)

The orchestrator contains the core business logic and processes each prompt in 4 phases.

Phase 4 → Finalization and Response Generation

The orchestrator assembles the final result, applies safety checks, saves the outcome, and records monitoring data.

1. Build the Final Response

- Combines all completed step outputs into one final answer
- Can fall back to a cached or default response if needed

2. Normalize the Output

- Returns a consistent output format, including user-facing text and optional JSON

3. Apply Safety Checks

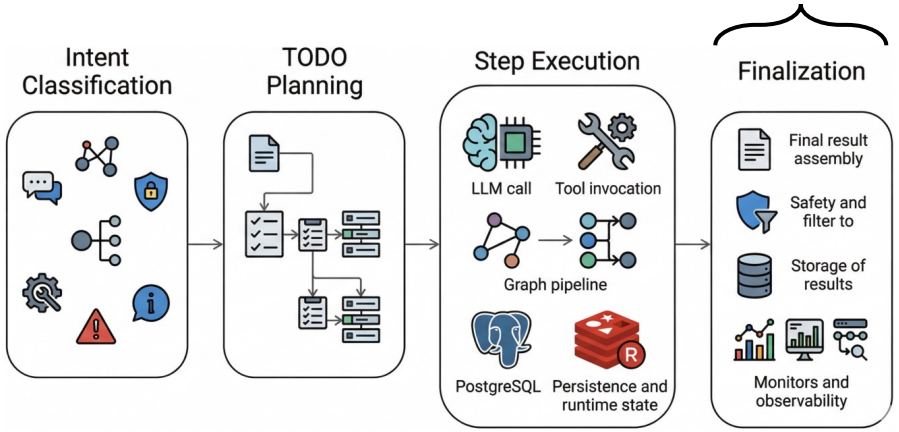
- Removes sensitive data and filters unsafe content before sending the response

4. Save the Final State

- Stores the final status, output, and metrics in PostgreSQL for audit and analysis

5. Emit Observability Data

- Records metrics and traces for dashboards, debugging, alerting, and monitoring




```
{
"goal": "Find collaborating institutions",
"steps": [step1, step2, step3],
"outputs": ["Found 5 institutions..."],
"todos": [completed_todo1, completed_todo2],
"metrics": { "duration_ms": 2340, "tokens": 1850 }
}
```

LLM PROVIDERS → MODEL-AGNOSTIC ARCHITECTURE

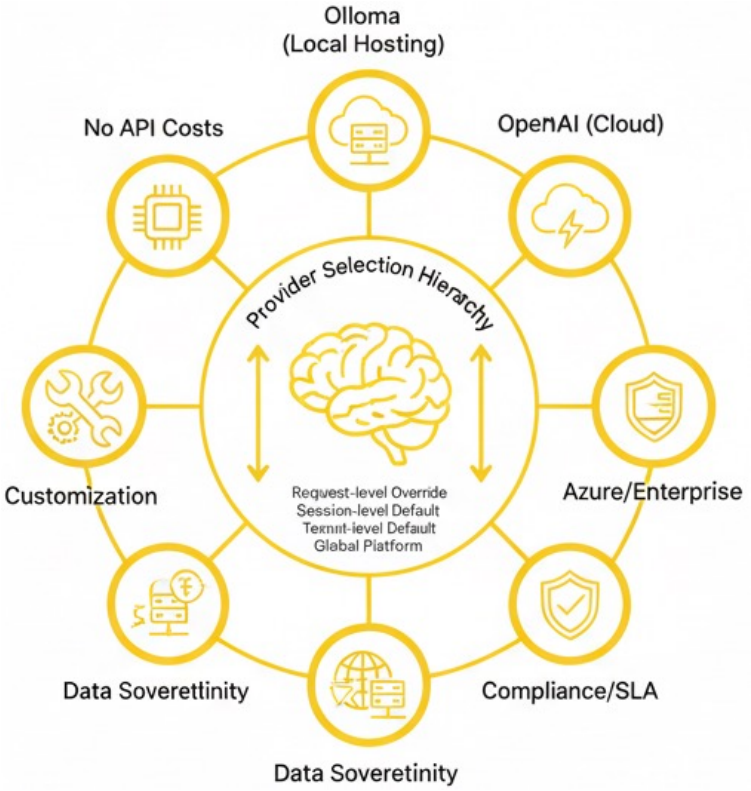
The platform can use multiple LLM providers, both local and cloud, without changing the application logic.

Local Models → Ollama
- Supports running open models locally on your own infrastructure
- Useful for data control, lower cost, offline use, and custom setups
- Example models: Phi-3, Mistral, LLaMA, Qwen, Gemma

Cloud Models → OpenAI / Azure / Others
- Supports hosted providers for strong model quality and managed infrastructure
- Useful for easy deployment, enterprise compliance, and regional hosting
- Examples: OpenAI, Azure OpenAI, and compatible providers

Provider Selection
- A model can be chosen at different levels:
- This gives flexibility to users, teams, and platform administrators

> Request-level override
> ↓ (if not set)
> Session-level default
> ↓ (if not set)
> Tenant-level default
> ↓ (if not set)
> Global platform default



MCP RUNTIME & TOOLS - 34 Tools, 12 Categories

Every MCP tool the agent can use is registered, validated, permission-checked, and audited.

Core Ideas

- Tool definitions are stored in a registry with schema, required scopes, and limits
- Each tool call is checked against user permissions before execution

Tool Invocation Flow

Request → validate input → check permissions → execute tool → audit → return result

Main Categories

- Graph → query Memgraph, inspect schema, generate or explain Cypher
- Security → inspect identity, permissions, and allowed actions
- System → check platform health, metrics, configuration, and status
- Catalog → discover available resources, tools, and schemas
- Model → list, inspect, warm up, or switch LLMs
- Agent → access conversation context, history, and session state
- Analytics → run aggregations and prepare visualization data
- Admin → perform restricted database administration actions
- ETL / CRUD / Export → import data, update graph entities, and export results in formats such as CSV or JSON

Examples

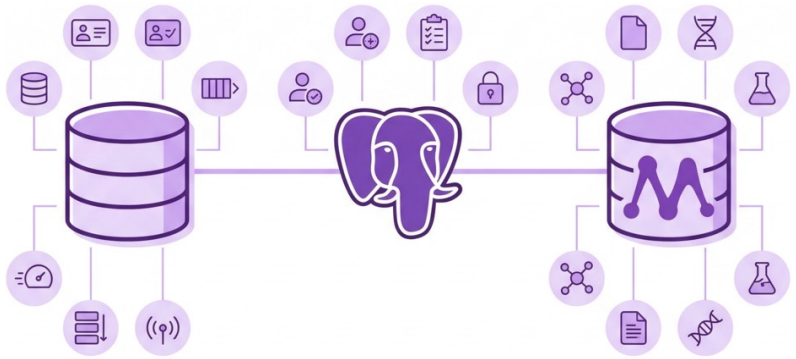
- Graph tools can answer a natural-language question by generating and running Cypher
- Security tools can tell a user what actions they are allowed to perform
- Export tools can return query results as CSV or JSON

DATA LAYER → REDIS, POSTGRESQL, MEMGRAPH

The platform uses three databases, each with a different role: speed, durability, and graph relationships.

Redis → Real-Time Runtime State for speed and real-time coordination
It stores:
- cache data, sessions, tenant config, and model defaults
- rate limits for users, tenants, and endpoints
- job queues, job state, and progress events
- cancellation flags, idempotency keys, and circuit-breaker state

PostgreSQL → Durable Platform State for reliable and transactional system data
It stores:
- tenants, sessions, runs, steps, and jobs
- job events, tool definitions, and tool invocation history
- model defaults, user preferences, and provider configs
- encrypted secrets, audit logs, and idempotency records



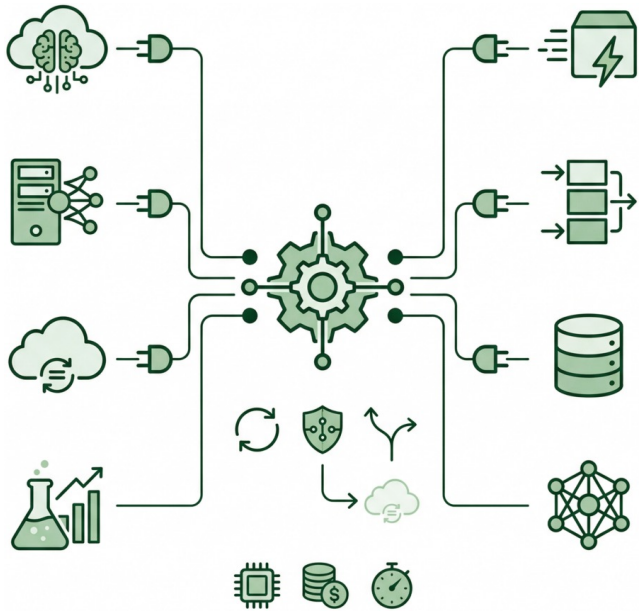
Memgraph → Bioinformatics Graph State for connected bioinformatics data and efficient Cypher queries
It stores:
- graph nodes such as users, institutions, tasks, files, datasets, samples, experiments, publications, genes, proteins, pathways, tools, workflows, and results
- graph relationships such as WORKS_AT, RUNS, INPUT, and OUTPUT
- node and edge metadata such as roles, status, timestamps, tags, and file details

ADAPTERS + RESILIENCE

The platform stays flexible and reliable by using swappable adapters and automatic failure handling.

Adapters

- The platform uses swappable adapters for both LLM providers and databases
- This keeps the business logic independent from specific vendors or storage systems
- For LLMs, the same internal interface can work with Ollama, OpenAI, Azure OpenAI, or stub/demo providers
- For databases, the same logic can call cache, queue, relational, or graph operations through Redis, PostgreSQL, and Memgraph adapters



Why This Matters

- Providers or backends can be changed without rewriting the orchestrator
- Example: switching from OpenAI to Azure only changes the adapter, not the rest of the code
- The same idea applies to database access such as cache.get() or graph.query()

Resilience

- The platform does not fail immediately when a provider has problems
- It uses retries with backoff, circuit breakers, and fallback providers
- If one provider becomes unhealthy, requests can fail fast or move to another provider
- Circuit breakers move between closed, open, and half-open states to control recovery

Operational Tracking

- Every LLM call is measured for tokens, cost, and latency
- This supports monitoring, budgeting, and capacity planning

BACKGROUND TASKS + OBSERVABILITY

The platform runs scheduled maintenance tasks and collects traces & metrics for visibility, reliability, & recovery.

Background Tasks

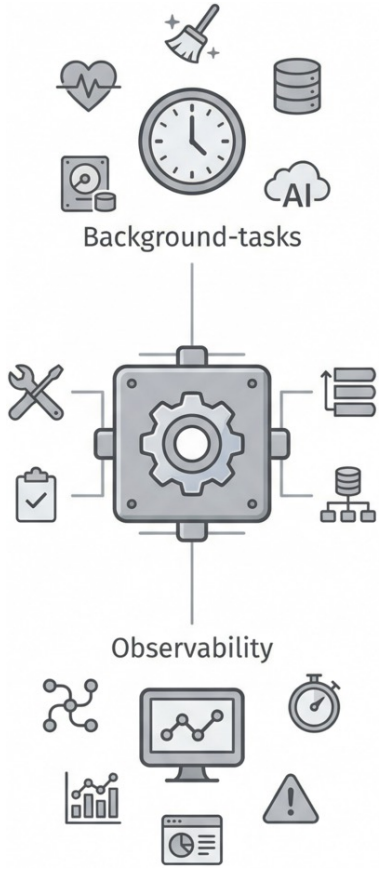
- APScheduler runs scheduled jobs automatically in the background.
- Health checks regularly verify PostgreSQL, Redis, Memgraph, and LLM providers.
- Cleanup jobs remove expired sessions, old cache entries, completed jobs, and orphaned runs.
- Daily backups protect PostgreSQL, Redis, Memgraph, and audit data.
- Provider monitoring updates health status, circuit breakers, and metrics.

Observability

- OpenTelemetry traces each request from end to end, including LLM calls, tool invocations, database queries, and job execution.
- Prometheus collects metrics from both the app and workers, including HTTP traffic, runs, steps, tools, jobs, LLM usage, circuit breakers, and system health.
- Grafana dashboards turn these traces and metrics into views of performance, failures, queue depth, provider health, costs, and overall system status.

Why This Matters

- Scheduled background tasks help keep the platform healthy, clean, and recoverable.
- Traces and metrics make it easier to diagnose slow requests, failing tools, unhealthy providers, and system-wide issues.



COMPARISON WITH STATE OF THE ART

Comparison of the CINECA Agentic Platform against Best SOTA platforms

Platform	Platform type	Orchestrator	MCP Tools	Governance	Graph	Monitoring	Model choice	Price	Main value	Why CAP is broader
CINECA Agentic Platform	Full-stack agent platform	✓	✓	✓	✓	✓	✓	Free	Secure enterprise orchestration for cases where users need one platform for agents, jobs, tools, and graph access	CAP is the only one here that combines all major layers in one system
Temporal	Workflow engine	✓	✗	✓	✗	✓	✓	Free	Best for durable backend workflows when retries, recovery, and long-running execution matter most	CAP adds agent planning, MCP-based tooling, and graph-native execution
LangGraph	Agent framework	✓	✗	✗	✗	✗	✓	Free	Best for building custom stateful agents when multi-step reasoning is the main goal	CAP adds security, jobs, UI, and operational platform layers
OpenAI Agents SDK	Agent SDK	✓	✗	✗	✗	✓	✗	Expensive	Best for quickly building tool-using agents when speed and simplicity matter most	CAP adds durability, multi-tenant governance, and graph support
Semantic Kernel	AI middleware	✓	✗	✗	✗	✗	✓	Free	Best for embedding AI into existing enterprise systems when orchestration is only one part of a bigger application	CAP includes the runtime and platform stack that SK leaves to the host app
n8n	Workflow automation platform	✓	✗	✗	✗	✗	✓	Cheap	Best for visual automation and integrations when business workflows matter more than deep agent reasoning	CAP adds true agent orchestration, stronger controls, and secure graph support

COMPARISON WITH STATE OF THE ART

The CINECA Agentic Platform is more complete than typical libraries.

How CAP Differs:

- It provides a full stack, including UI, APIs, jobs, and orchestration
- It includes durable execution with retries, checkpoints, and background processing
- It supports an agent planning loop with MCP-based tool execution
- It adds strong security and governance through JWT, RBAC, tenancy, and output/input guards
- It includes built-in NL→Cypher support with validation and tenant isolation
- It also includes observability, LLM-provider flexibility, and production controls

How Other Platforms Compare:

- Temporal and Argo are strong for workflow durability, but they are not agent platforms
- LangGraph, OpenAI Agents SDK, Semantic Kernel, LlamaIndex, and Haystack support agent logic, but usually require extra work for security, durability, and platform operations
- n8n and Windmill provide useful workflow UIs, but they are not designed as secure agent orchestration platforms

Main Takeaway

CAP stands out because it combines full-stack delivery, durable orchestration, governed tool execution, secure graph querying, and observability in one platform, while most alternatives are strong only in some of these areas

CONCLUSION + FUTURE WORK

Closed the production gap.

What Was Achieved

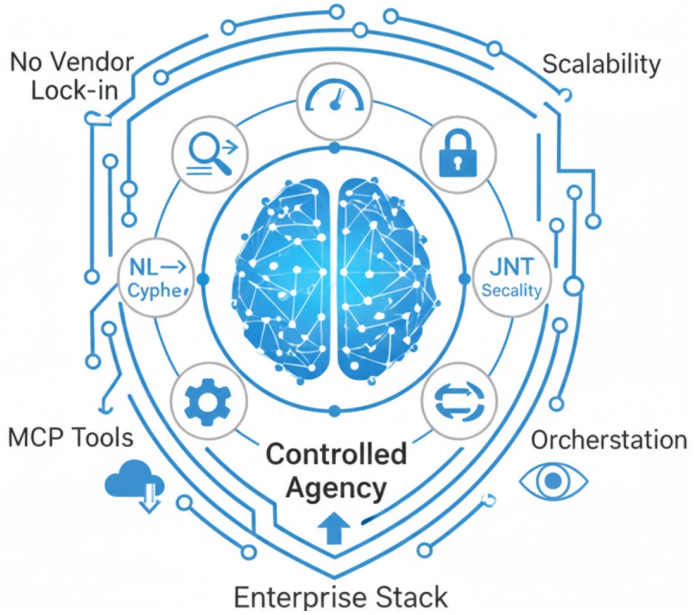
- The platform closes the production gap by turning natural language into safe, traceable graph actions
- It delivers a full enterprise stack with secure UIs, APIs, orchestration, workers, and a multi-database backend
- Agent runs are controlled, not just conversational, with MCP tools, RBAC, tenant isolation, and safe NL→Cypher execution
- It is production-ready, with large API coverage, governed tools, resilience mechanisms, and built-in observability
- It remains flexible across providers such as OpenAI, Azure, and Ollama

Key Contribution

- A secure and extensible NL-to-action platform ready to grow across tools, models, and tenants

Future Work

- Add an “Ask Mode” for lightweight question answering without full agent planning
- Make the orchestrator handle more prompt types more reliably, from simple to ambiguous or multi-step requests
- Improve recovery and reproducibility through stronger checkpointing and deterministic re-runs
- Add smarter scheduling with priorities, quotas, and backpressure under heavy load



Thanks For Your Attention

Feel free to ask any question